

Hash-Based Blind Signatures: First Steps

Javier Herranz¹ and Hugo Louiso²

¹ Dept. Matemàtiques
Universitat Politècnica de Catalunya-BarcelonaTech
c. Jordi Girona 1-3, 08034, Barcelona, Spain
javier.herranz@upc.edu
² Université de Bordeaux
Bordeaux, France
hugolouiso@gmail.com

Abstract. Hash-based signatures are a strong candidate for post-quantum scenarios requiring authentication and integrity. Their security relies only on (well-studied) properties of hash functions, so they may be thought as being more robust than other schemes that (today) resist quantum attacks, like those based on lattices, coding or isogenies.

Recent works also study hash-based signature schemes with additional properties, such as group, ring, threshold, or aggregate signature schemes. In this work, we do the same for the important case of blind signatures. We describe a secure (in the programmable random oracle model) modification of Fischlin’s generic construction of blind signatures that can be instantiated with different hash-based protocols. As a result, we obtain the first blind signature schemes whose security relies only on the collision resistance of hash functions. Finally, as a proof of concept, we have implemented one (highly non-optimized) of the possible instantiations.

Keywords: hash-based signature schemes, blind signatures, post-quantum security, zero-knowledge proofs, MPC-in-the-Head.

1 Introduction

In a future time when some companies, organizations and maybe even citizens will have access to powerful enough quantum computers, some of the cryptographic paradigms that are used today will become insecure, because of the capability (thanks to Shor’s algorithm [38]) of solving in polynomial time problems like RSA or the Discrete Logarithm problem in cyclic groups.

For this reason, the cryptographic community is actively working on the proposal, analysis and standardization of new cryptographic protocols that remain secure even in the presence of quantum computers. For instance, the NIST is organizing several such processes, the first one for public key encryption and digital signatures. The proposals that, today, seem to be more promising are based on Euclidean lattices, or coding, or isogenies between elliptic curves, or based on properties of hash functions.

Regarding hash-based cryptography, it is not possible to build hash-based public key encryption schemes, but it is possible to design provably secure hash-based signature schemes; for instance, one of the digital signature schemes selected by NIST is SPHINCS+ [10,11], whose security depends only on (well-known) properties of the underlying hash functions. Some people argue that relying the (post-quantum) security of a cryptographic scheme on

properties of a hash function is more robust than relying it on the hardness of some mathematical problem, related for instance to lattices (such as the LWE and ring LWE problems), because it seems more plausible that, in the future, somebody will find a clever quantum algorithm to solve such mathematical problems. In contrast, breaking a hash function, even with a quantum computer, seems very unlikely; this is the same situation as with symmetric encryption (like AES). It is believed that such symmetric primitives (block ciphers, hash functions) will generically remain secure in the post-quantum era, as long as the length of their parameters is increased from time to time.

There are many extensions of the plain notion of digital signature, with applications in different real-life scenarios: ring, group, aggregate, threshold, blind, attribute-based signatures, to name a few. In most of the cases, designing such advanced signature schemes is possible thanks to the algebraic structure of the underlying mathematical tools, like cyclic groups, bilinear pairings or (in the post-quantum regime) lattices. In the hash-based setting, there are direct constructions for some of these extensions, for instance group signatures [3] and threshold signatures [32], but not for other extensions, due to the lack of algebraic structure of hash functions.

However, in some cases there exist generic constructions of advanced signature schemes that require the combination of cryptographic primitives, like commitment schemes, signature schemes or (non-interactive) zero-knowledge proofs of knowledge, that can be securely implemented with hash functions. A specific comment is mandatory for the case of non-interactive zero-knowledge proofs (NIZKs), which exist in the random oracle model for any language to be proved, but which were known to be efficient only for languages with some efficient algebraic structure (for instance, proving knowledge of x such that $g^x = y$, for public elements g, y in a cyclic group $\mathbb{G}$). This did not include statements related to hash functions, whose algebraic description is very inefficient, for instance proving knowledge of x such that $H(x) = y$, for some public y and some public hash function H . However, the situation has changed a lot in the last years, with the development of new techniques for designing secure (and reasonably efficient) NIZK proofs related to any circuit. Whereas the techniques that lead to the shortest NIZK proofs, SNARKs, rely on bilinear pairings and thus are not post-quantum secure, there are other techniques whose security is either unconditional or relies only on properties of hash functions. Some examples are NIZKs resulting from the MPC-in-the-head paradigm [29,30], STARKs [8] and other similar systems like Ligerio [2] or Aurora [7].

Summing up, proving in zero-knowledge the knowledge of some x such that $H(x) = y$ for a public hash function H and a public output y is not considered to be inefficient anymore. This fact has allowed some authors to instantiate well-known generic constructions of some advanced digital signature schemes with hash-based primitives, leading to schemes with (post-quantum) security based only on well-known properties of hash functions. Specifically, group and ring signature schemes were proposed in [31], linkable ring signatures were proposed in [39], aggregate and threshold signatures were proposed in [33], aggregate and multisignatures were proposed in [21].

There is another kind of signatures where this approach can be followed: blind signatures [18]. In a blind signature scheme, a user U and a server S (with public key pk_S) interact such that, at the end, the user obtains a valid signature σ on its input message m , under public key pk_S , whereas the server does not obtain any information about the message m that it has (blindly) signed. These signatures are used, for instance, in systems for electronic coins or electronic voting. In particular, for electronic voting there are mainly two families of solutions: (i) one where vote secrecy is obtained through the combination of public key encryption and either mixing or homomorphic tally, (ii) another one where vote secrecy is

obtained through the use of some kind of signature scheme providing privacy or anonymity: blind signatures, (linkable) ring signatures, etc. In a hypothetical future where one knows polynomial time quantum algorithms to solve any lattice / isogeny / coding problem, and where the only quantum-resistant cryptography is symmetric cryptography (block ciphers, hash functions), the only secure electronic voting systems will basically come from the use of hash-based ring or blind signature schemes.

More generally, the race for designing the best (shortest public keys and/or shortest signatures) blind signature scheme with presumable post-quantum security is really active in the last months, see for instance [15,27,35,4,19] and references therein. In this work we add one more possibility to get blind signature schemes with post-quantum security: hash-based blind signatures, which will not be as efficient as specific lattice or coding based blind signature schemes, but which may be considered more robust in terms of security.

1.1 Our Contribution and Technical Overview

Yes, indeed there are generic constructions of blind signature schemes, the main one due to Fischlin [23]; we propose to use a modification of Fischlin’s construction (also employed in the paper [4]), which can be instantiated basically with three ingredients: a commitment scheme, a plain signature scheme and a non-interactive zero-knowledge proof of knowledge (NIZK). The idea of the resulting blind signature schemes is the following one. The server S generates a pair of keys (sk, pk) for the plain signature scheme and publishes pk as its public key. Later, when a user U wants to obtain a signature by S on a (secret) message m , the following (two rounds) interaction is performed:

1. U chooses randomness r for the corresponding commitment scheme, and computes a commitment $M = \text{Com}(m; r)$ for m using randomness r . The commitment M is sent to S .
2. S uses its secret key sk to compute a plain signature Σ on the commitment M , and sends Σ to U .

After those two rounds of interaction, U can compute the final signature σ on m as a NIZK proof of knowledge of (r, Σ) that satisfies the following circuit C : the verification algorithm of the plain signature scheme outputs 1 when run with public key pk , message $\text{Com}(m; r)$ and signature Σ .

These three primitives can be securely instantiated with hash-based protocols:

- As the commitment scheme, we can either use $M = \text{Com}(m; r) = H_c(m \parallel r)$ for a hash function H_c , or for more security, we can use the Halevi-Micali [26] commitment scheme that combines such a function H_c with a (unconditionally secure) universal hash family.

- As the plain signature scheme, we can use a (stateful) tree-based (or Merkle) signature scheme, MSS for short, using some one-time signature (OTS, for short) scheme based on hashes in the leaves of the tree, or some extensions of MSS like XMSS [16] (also stateful) or SPHINCS+ [11] (which is stateless).

- As the NIZK for proving knowledge of a satisfying input for the resulting circuit C , we can use the proof that results from applying the Fiat-Shamir heuristic (with some hash function H_{FS}) to some ZK interactive protocol for that circuit, either in the family of MPC-in-the-head protocols [29,30] or in the family of STARKs [8] and related protocols; all the resulting NIZK proof systems have knowledge soundness (in the programmable random oracle model), which is the property that we need.

The resulting blind signature schemes are the first ones³ whose security depends only on security properties of hash functions.

1.2 Concurrent / Subsequent, and very Related, Work

As we have mentioned, the area of research in blind signature schemes with post-quantum security is really active in the last months. In particular, after the first publication of this work on the IACR ePrint repository, two other very related works have been published on the same repository:

- The work by Baum et al. [4] considers exactly the same modification of Fischlin’s generic construction of blind signatures as we do here. They instantiate the construction with primitives based on multivariate quadratic systems (like the MAYO signature scheme) with the main goal of obtaining short blind signatures and, especially, short running times for signature generation.
- The work by Ciampi et al. [19] proposes a different, and really smart, modification of Fischlin’s generic construction of blind signatures, a bit more complicated than ours (thus leading to a bit less efficient schemes). But from a theoretical point of view, they obtain stronger results than us: (i) they show that their generic construction can be instantiated with one-way functions only (as opposed to our collision-resistant hash functions), and (ii) the security of the resulting blind signatures can be proved in the Universal Composability framework (instead of our Game Based framework) and in the non-programmable random oracle model (instead of our stronger programmable random oracle model).

1.3 Organization of the Paper

In Section 2 we explain the ingredients that are used in the design of the new blind signature schemes. Then we devote Section 3 to the definitions of blind signatures, the generic construction that we consider in this work, and its security analysis. In Section 4 we discuss different ways to instantiate this generic construction with hash-based protocols. Therein, we also present our C implementation of one of the resulting schemes (far from being the most efficient possibility), which is described in detail in Appendix A for completeness. The paper is completed in Section 5, where we summarize our work and we discuss some extensions of it.

2 Preliminaries

In this section we review the ingredients that will appear in the generic construction of our blind signature schemes that we will consider later: signature schemes, commitment schemes and non-interactive zero-knowledge systems. To ease readability, we simplify some descriptions of cryptographic protocols, in particular by omitting their dependence on public parameters (in the case of commitment schemes and NIZKs, for instance).

³ Concurrently with those in [19].

2.1 Digital Signatures

A signature scheme $S = (S.\text{KeyGen}, S.\text{Sign}, S.\text{Vfy})$ consists of three polynomial-time algorithms. The key generation algorithm $S.\text{KeyGen}(1^\lambda) \rightarrow (\text{sk}, \text{pk})$ takes as input a security parameter λ and outputs a pair of secret key sk and public key pk . The signing algorithm $S.\text{Sign}(m, \text{sk}) \rightarrow \sigma$ takes as input a message m and a secret key sk and outputs a signature σ . Finally, the verification algorithm $S.\text{Vfy}(m, \sigma, \text{pk}) \rightarrow b$ takes as input a message m , and signature σ and a public key pk and outputs a bit $b \in \{0, 1\}$, with $b = 1$ meaning “valid signature” and $b = 0$ meaning “invalid signature”.

For security, a signature scheme is secure, specifically unforgeable under chosen message attacks, if any polynomial-time adversary $\mathcal{F}$ has negligible probability in obtaining a pair (m^*, σ^*) such that $S.\text{Vfy}(m^*, \sigma^*, \text{pk}) = 1$, when given as input the public key pk and given access to a signing oracle (which, on input a message m_i sent by $\mathcal{F}$, sends back $\sigma_i \leftarrow S.\text{Sign}(m_i, \text{sk})$, sk being the secret key matching public key pk). Of course, $\mathcal{F}$ succeeds only if (m^*, σ^*) is different to any pair (m_i, σ_i) obtained from the signing oracle. In case the signature scheme is stateful, the security definition is modified so that the signing oracle maintains the state of the signing system.

2.2 Commitment Schemes

A commitment scheme Com basically consists of a function $\text{Com}(m; r) \rightarrow M$ that one applies to some message m by picking uniform randomness $r \leftarrow_R \{0, 1\}^{p(\lambda)}$ from a set of size exponential in the security parameter λ .

A commitment scheme is useful when it satisfies two properties: being hiding and binding.

Definition 1. A commitment scheme Com is hiding when $\frac{1}{2}$ minus the probability that $b' = b$ in the game below is a negligible function of λ for any PPT two-stage adversary $\mathcal{A}_h = (\mathcal{A}_{h_1}, \mathcal{A}_{h_2})$:

$$\begin{aligned} (m_0, m_1, st) &\leftarrow \mathcal{A}_{h_1}(1^\lambda) \\ b &\leftarrow_R \{0, 1\} \\ r &\leftarrow_R \{0, 1\}^{p(\lambda)} \\ b' &\leftarrow \mathcal{A}_{h_2}(st, \text{Com}(m_b; r)) \end{aligned}$$

Definition 2. A commitment scheme Com is binding when the probability that $\tilde{c}_1 = \tilde{c}_2$ and $m_1 \neq m_2$ in the game below is a negligible function of λ for any PPT adversary $\mathcal{A}_b$:

$$\begin{aligned} (m_1, r_1, m_2, r_2) &\leftarrow \mathcal{A}_b(1^\lambda) \\ \tilde{c}_1 &\leftarrow \text{Com}(m_1; r_1) \\ \tilde{c}_2 &\leftarrow \text{Com}(m_2; r_2) \end{aligned}$$

2.3 NIZK Proofs

We will define the protocols and security of a non-interactive zero-knowledge proof system for the particular case where the statement to be proven is the knowledge of an input that makes some (arithmetic) circuit output 1.

Let C be a circuit that maps a (long) input to a bit, with output 1 meaning that the input satisfies the circuit C , and output 0 meaning that it does not. The input is split into a public part γ and a secret part (or witness) ω . We will denote an evaluation of the circuit with this global input as $C(\gamma; \omega)$.

A non-interactive zero-knowledge (NIZK) proof system Π for circuit C is a protocol to prove, given some public input γ , the knowledge of some witness ω such that $C(\gamma; \omega) = 1$, without revealing anything else about ω .

The NIZK system $\Pi = (\Pi.\text{Prove}^\mathcal{O}, \Pi.\text{Vfy}^\mathcal{O})$ consists of two algorithms, both of them having as part of their inputs the description of the circuit C and its public partial input γ . Furthermore, the two algorithms have access to the same cryptographic oracle $\mathcal{O}$; typically, this is a hash function modeled as a (programmable) random oracle.

- $\Pi.\text{Prove}^\mathcal{O}(C, \gamma, \omega) \rightarrow \pi$ takes as additional input a witness ω , in principle satisfying $C(\gamma; \omega) = 1$, and outputs a proof π .
- $\Pi.\text{Vfy}^\mathcal{O}(C, \gamma, \pi) \rightarrow d \in \{0, 1\}$ takes as inputs the description of the circuit C , its public input γ and a proof π , and outputs a bit d , with $d = 1$ meaning that the proof is accepted, and $d = 0$ meaning that the proof is rejected.

The first property required to a NIZK system is that of correctness: if the secret input ω of $\Pi.\text{Prove}$ satisfies $C(\gamma; \omega) = 1$, then the proof π resulting from the Prove algorithm is always accepted as valid by the Verification algorithm.

The other two properties are zero-knowledge and soundness. Intuitively, zero-knowledge means that the output π does not leak information about the secret witness ω , other than the fact that $C(\gamma; \omega) = 1$.

Definition 3. A NIZK system Π enjoys the zero-knowledge property if there exists a PPT algorithm Sim , called simulator (who can control the oracle $\mathcal{O}$), such that $\frac{1}{2}$ minus the probability that $b' = b$ in the game below is a negligible function of λ for any PPT adversary $\mathcal{A}_{ZK}$ and any inputs γ, ω for C such that $C(\gamma; \omega) = 1$:

$$\begin{aligned} \pi_0 &\leftarrow \Pi.\text{Prove}^\mathcal{O}(C, \gamma, \omega) \\ \pi_1 &\leftarrow \text{Sim}(C, \gamma) \\ b &\leftarrow_R \{0, 1\} \\ b' &\leftarrow \mathcal{A}_{ZK}^\mathcal{O}(\pi_b, C, \gamma) \end{aligned}$$

Soundness means that it is impossible (for a dishonest prover) to obtain a proof π such that $\Pi.\text{Vfy}(C, \gamma, \pi) = 1$ without knowing a witness ω such that $C(\gamma; \omega) = 1$. The stronger knowledge soundness property requires that such witness ω^* can be extracted from the execution of such an accepted proof.

Definition 4. A NIZK system Π enjoys the knowledge soundness property if there exists a PPT algorithm Ext , called extractor, such that the probability that $\Pi.\text{Vfy}(C, \gamma, \pi^*) = 1$ and $C(\gamma; \omega) \neq 1$ is a negligible function of the security parameter λ for any PPT adversary $\mathcal{A}_{ks}$ and any public input γ for circuit C :

$$\begin{aligned} \pi &\leftarrow \mathcal{A}_{ks}^\mathcal{O}(C, \gamma) \\ \omega &\leftarrow \text{Ext}(C, \gamma, \pi, Q) \end{aligned}$$

(Here Q denotes the set of all query-response pairs obtained by $\mathcal{A}_{ks}$ by interacting with oracle $\mathcal{O}$.)

A typical way of designing NIZK proof systems for some circuit is to first design an interactive proof system, where the prover and the verifier exchange some messages, and then to apply the Fiat-Shamir transformation, which replaces the messages by the verifier (as long as they all consist of random and uniform messages) with outputs of a hash function H_{FS} applied to the previous messages exchanged in the interaction. If the interactive protocol is (special) sound and zero-knowledge, then the resulting non-interactive protocol is also secure, in the random oracle model for the hash function H_{FS} . The knowledge soundness, in this case, is proved through a rewinding strategy, which requires that the extractor Ext programs the random oracle that models H_{FS} .

3 Blind Signatures: Definitions and Generic Construction

Since the blind signature schemes resulting from Fischlin's generic transformation [23] and its modifications are two-round (optimal) schemes, we are going to give the definitions of the protocols and security properties of blind signature schemes directly adopted to this case of optimal two-round schemes, as done in other works like [14,4].

A blind signature scheme $\text{BS} = (\text{BS.KG}, \text{BS.Sig}_1, \text{BS.Sig}_2, \text{BS.Sig}_3, \text{BS.Vfy})$ is a set of five algorithms that work as follows:

- $\text{BS.KG}(1^\lambda) \rightarrow (\text{sk}_{\text{BS}}, \text{pk}_{\text{BS}})$ is the key generation algorithm, that takes as input the security parameter λ and outputs a pair of secret and public keys for the signer.
- $\text{BS.Sig}_1(\text{pk}_{\text{BS}}, m) \rightarrow (\text{st}, M)$ is the first round of the interaction, run by the user, with secret input a message m ; the user stores the state st and sends the value M to the signer.
- $\text{BS.Sig}_2(\text{sk}_{\text{BS}}, M) \rightarrow \Sigma$ is the second round of the interaction, run by the signer, with secret input sk_{BS} . The resulting value Σ is sent back to the user, finishing the interaction between signer and user.
- $\text{BS.Sig}_3(\Sigma, \text{st}) \rightarrow \sigma$ is the final computation run by the user, who outputs σ as the (blind) signature on message m . We stress that the output σ can be defined as $\perp$ if this algorithm fails.
- $\text{BS.Vfy}(m, \sigma, \text{pk}_{\text{BS}}) \rightarrow d \in \{0, 1\}$ is the typical verification algorithm of any signature scheme, that outputs bit $d = 1$ if and only if σ is accepted as a valid signature for message m under public key pk_{BS} .

3.1 Security Definitions

We omit for brevity the formal definition of the first notion that must be satisfied by a blind signature scheme, that of correctness; intuitively, if all the algorithms are run as established by the scheme, then the verification algorithm must always output 1.

The following property is that of blindness: a signer must not be able to link a valid pair (m, σ) and the specific interaction with the user that has led to it; this must hold even if the signer knows that the correct interaction is one between two possible ones (indistinguishability notion).

Definition 5. A blind signature scheme BS enjoys the blindness property when $\frac{1}{2}$ minus the probability that $b' = b$ in the game below is a negligible function of λ for any PPT stateful three-stage adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3)$ and any pair of messages m_0, m_1 :

$$\begin{aligned} \text{pk}_{\text{BS}} &\leftarrow \mathcal{B}_1(1^\lambda) \\ b &\leftarrow_R \{0, 1\} \\ (\text{st}_0, M_0) &\leftarrow \text{BS.Sig}_1(\text{pk}_{\text{BS}}, m_0) \\ (\text{st}_1, M_1) &\leftarrow \text{BS.Sig}_1(\text{pk}_{\text{BS}}, m_1) \\ (\Sigma_b, \Sigma_{1-b}) &\leftarrow \mathcal{B}_2(M_b, M_{1-b}) \\ \sigma_0 &\leftarrow \text{BS.Sig}_3(\Sigma_0, \text{st}_0) \\ \sigma_1 &\leftarrow \text{BS.Sig}_3(\Sigma_1, \text{st}_1) \\ \text{If } \perp \in \{\sigma_0, \sigma_1\}, \text{ then } (\sigma_0, \sigma_1) &\leftarrow (\perp, \perp) \\ b' &\leftarrow \mathcal{B}_3(\sigma_0, \sigma_1) \end{aligned}$$

Finally, the one-more unforgeability notion requires that a user must not be able to obtain more valid blind signatures than the number of successful interactions that it had run with the signer.

Definition 6. A blind signature scheme BS enjoys the one-more unforgeability property when the probability that all messages in $\{m_1, \dots, m_{q+1}\}$ are pairwise different and $\text{BS.Vfy}(m_i, \sigma_i, \text{pk}_{\text{BS}}) = 1$ for all $i \in \{1, \dots, q+1\}$, in the game below, is a negligible function of λ for any PPT forger $\mathcal{F}$ that makes at most q queries to a signing oracle for BS.Sig_2 :

$$\begin{aligned} (\text{sk}_{\text{BS}}, \text{pk}_{\text{BS}}) &\leftarrow \text{BS.KG}(1^\lambda) \\ \{(m_i, \sigma_i)\}_{1 \leq i \leq q+1} &\leftarrow \mathcal{F}^{\text{BS.Sig}_2(\text{sk}_{\text{BS}}, \cdot)}(\text{pk}_{\text{BS}}) \end{aligned}$$

In case the blind signature scheme is stateful, the signing oracle in the security definition maintains the state of the scheme when answering the queries by $\mathcal{F}$.

3.2 A Generic Construction

We consider in this work the following modification⁴ of the generic construction of blind signatures due to Fischlin [23].

Let Com be a commitment scheme, let $S = (S.\text{KeyGen}, S.\text{Sign}, S.\text{Vfy})$ be a standard signature scheme, and let $\Pi = (\Pi.\text{Setup}, \Pi.\text{Prove}, \Pi.\text{Vfy})$ be a non-interactive proof system for the language defined by secret witness $\omega = (r, \Sigma)$, public inputs m, pk and circuit C such that $C(\omega) = 1$ if and only if $S.\text{Vfy}(\text{Com}(m; r), \Sigma, \text{pk}) = 1$.

Then the following blind signature scheme BS can be considered:

- $\text{BS.KG}(1^\lambda) \rightarrow (\text{sk}_{\text{BS}}, \text{pk}_{\text{BS}})$ works by first calling $S.\text{KeyGen}(1^\lambda) \rightarrow (\text{sk}, \text{pk})$ and then defining $\text{sk}_{\text{BS}} = \text{sk}$ and $\text{pk}_{\text{BS}} = \text{pk}$.
- $\text{BS.Sig}_1(\text{pk}_{\text{BS}}, m) \rightarrow (\text{st}, M)$ works by uniformly taking randomness $r \leftarrow_R \{0, 1\}^{p(\lambda)}$ and computing $M = \text{Com}(m; r)$; the state is defined as $\text{st} = (m, r)$.
- $\text{BS.Sig}_2(\text{sk}_{\text{BS}}, M) \rightarrow \Sigma$ consists in signing message M with signature scheme S , that is, running $\Sigma \leftarrow S.\text{Sign}(M, \text{sk}_{\text{BS}})$.
- $\text{BS.Sig}_3(\Sigma, \text{st}) \rightarrow \sigma$ outputs $\sigma = \perp$ if $S.\text{Vfy}(M, \Sigma, \text{pk}_{\text{BS}}) \neq 1$. Otherwise, it defines $\sigma = \pi \leftarrow \Pi.\text{Prove}(C, (\text{pk}_{\text{BS}}, m); (r, \Sigma))$.
- $\text{BS.Vfy}(m, \sigma, \text{pk}_{\text{BS}}) \rightarrow d \in \{0, 1\}$ works by defining the bit d as the output bit obtained when running $\Pi.\text{Vfy}(C, (\text{pk}_{\text{BS}}, m); \sigma)$.

Security Analysis The correctness of the blind signature scheme follows directly from the correctness of the primitives used.

We include here (non-detailed) proofs of the blindness and one-more unforgeability of the blind signature scheme resulting from this generic construction. This is done for the sake of completeness, because these results are today considered as a kind of well-known folklore in this area. In particular, exactly the same security results are proven in Theorem 1 of [4]. The security proofs for other modifications of Fischlin's generic construction [23], for instance, those in [14], are also very similar.

Theorem 1. Assuming that Com enjoys the hiding property and that Π enjoys the zero-knowledge property, the blind signature scheme BS enjoys the blindness property.

Proof. (Sketch.) Let $\mathcal{G}_0$ be the blindness game (in Definition 5) played by a challenger and an adversary $\mathcal{B} = (\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3)$. We modify this game, which results in game $\mathcal{G}_1$, by replacing steps

$$\sigma_0 \leftarrow \text{BS.Sig}_3(\Sigma_0, \text{st}_0) = \Pi.\text{Prove}(C, (\text{pk}_{\text{BS}}, m_0); (r_0, \Sigma_0))$$

⁴ Exactly the same modification is used in [4].

$$\sigma_1 \leftarrow \text{BS.Sig}_3(\Sigma_1, \text{st}_1) = \Pi.\text{Prove}(\mathcal{C}, (\text{pk}_{\text{BS}}, m_1); (r_1, \Sigma_1))$$

with two executions of the simulator Sim of Π (see Definition 3), namely $\text{Sim}(\mathcal{C}, (\text{pk}_{\text{BS}}, m_0))$ and $\text{Sim}(\mathcal{C}, (\text{pk}_{\text{BS}}, m_1))$, respectively, which can actually be run by the adversary $\mathcal{B}$ itself.

Due to the zero-knowledge property of Π , the behaviour of the blindness adversary $\mathcal{B}$ in games $\mathcal{G}_0$ and $\mathcal{G}_1$ must be essentially the same.

But in game $\mathcal{G}_1$, the only input that adversary $\mathcal{B}$ gets are $M_b = \text{Com}(m_b; r_b)$ and $M_{1-b} = \text{Com}(m_{1-b}; r_{1-b})$. Clearly, if $\mathcal{B}$ could guess the bit b with probability significantly greater than $\frac{1}{2}$, then the hiding property of Com could be broken.

Putting together the statements in the two previous paragraphs, one concludes that the zero-knowledge property of Π and the hiding property of $\mathcal{C} \Downarrow$ imply the blindness of the constructed blind signature scheme BS . $\square$

Theorem 2. *Assuming that Com enjoys the binding, that S enjoys the unforgeability property and that Π enjoys the knowledge soundness property, the blind signature scheme BS enjoys the one-more unforgeability property.*

Proof. (Sketch.) Let us assume the existence of a successful forger $\mathcal{F}_{\text{BS}}$ against the one-more unforgeability of BS , and construct a forger $\mathcal{F}_{\text{S}}$ against the underlying signature scheme S .

Every query M_i , for $i \in \{1, 2, \dots, q\}$ that $\mathcal{F}_{\text{BS}}$ makes to signing oracle BS.Sig_2 , the query is forwarded by $\mathcal{F}_{\text{S}}$ to its own signing oracle; the obtained signature Σ_i is sent back from $\mathcal{F}_{\text{S}}$ to $\mathcal{F}_{\text{BS}}$.

When $\mathcal{F}_{\text{BS}}$ outputs $q + 1$ valid pairs $\{(m_i, \sigma_i)\}_{1 \leq i \leq q+1}$, with the m_i 's being pairwise different, since it holds $\Pi.\text{Vfy}(\mathcal{C}, (\text{pk}_{\text{BS}}, m_i); \sigma_i) = 1$, we can use the knowledge soundness property of Π to run $q + 1$ times the extractor $\text{Ext}(\mathcal{C}, (\text{pk}_{\text{BS}}, m_i), \sigma_i, Q) \rightarrow (r_i, \Sigma_i)$ such that (r_i, Σ_i) is a valid witness, that is, $\text{S.Vfy}(\text{Com}(m_i; r_i), \Sigma_i, \text{pk}_{\text{BS}}) = 1$.

This means that a valid standard signatures of messages $M_1, M_2, \dots, M_{q+1}$ are obtained, where $M_i = \text{Com}(m_i; r_i)$ for $i \in \{1, 2, \dots, q+1\}$. If there are $\leq q$ different messages in the set $\{M_1, \dots, M_{q+1}\}$, then there would be some collision in the computation of the commitments, and so the binding property of Com would be broken, which contradicts one of the hypotheses of the theorem. Therefore, the $q + 1$ elements $M_1, M_2, \dots, M_{q+1}$ are pairwise different and this means that $\mathcal{F}_{\text{S}}$ has obtained $q + 1$ valid signatures for S after making q signing queries, which is a successful attack against the unforgeability of S . $\square$

We note that the oracle of the NIZK system Π is a hash function H_{FS} that is modeled as a random oracle model. Crucially, this hash function does not appear in the circuit $\mathcal{C}$ and thus H_{FS} can be thought of as a black box object to which we can (without any paradox) apply the random oracle heuristic. The key point is that the security of the other primitives (commitment scheme Com and signature scheme S) must be provable without relying on the random oracle model for any hash function that appears in the circuit $\mathcal{C}$.

4 Hash-Based Blind Signatures

In this section we discuss possible hash-based instantiations of the generic construction described in previous Section 3.2, by giving some possible choices for the three ingredients that appear in the construction.

One of the choices (essentially the same as in [19]) leads to the most efficient scheme, whereas another option leads to a very inefficient scheme; this last option, however, is the one we dared to implement (due to our limited implementation skills), resulting in the proof of concept that we describe in Section 4.3 and Appendix A.

4.1 Possible Instantiations

Choices for the Commitment Com. The easiest hash-based commitment scheme is $\text{Com}(m; r) = H_c(m \parallel r)$, for some hash function H_c . Collision resistance of H_c directly implies the binding property for Com. However, there is no standard property of hash functions (collision resistance, first or second preimage resistance) that implies the hiding property for Com. One possibility, typical in the area, is to establish the heuristic assumption that H_c satisfies the hiding property (such an assumption is weaker than assuming that H_c behaves as a random oracle).

Another possibility is to use the commitment scheme proposed by Halevi and Micali [26], which combines a hash function H_c and a universal hash family, and which enjoys the hiding and binding property in a provable way, assuming only the collision resistance of the hash function H_c .

Choices for the Signature Scheme S. We need to use a hash-based signature scheme whose security is proved in the standard model for the employed hash functions. The first and simpler candidate is the basic Merkle signature scheme, described in [20], and then other subsequent schemes with improved security and efficiency, like XMSS [16] or SPHINCS(+) [9,10]. While the two first ones are stateful signature schemes, SPHINCS(+) is stateless. All of them use as an ingredient a hash-based one-time signature scheme [28]. Interesting results on optimizing the efficiency of hash-based one-time signatures have been recently published [34].

We insist that hash-based signature schemes whose security is proved in the random oracle model cannot be used in our construction, to avoid the paradox of interpreting a same hash function as a random oracle and as a specific function that forms part of circuit C. This rules out the possibility of using very efficient signature schemes like FAEST [5].

Choices for the NIZK System II. The amount of available hash-based NIZK systems with the required properties and with good efficiency has significantly increased in the last years. There are essentially two families of candidates:

- Those derived from the MPC-in-the-Head paradigm [29,30] and its derivatives / improvements, as VOLE-in-the-head [6] or Threshold-Computation-in-the-head [22]. In particular, the Ligerio [2] and Ligerio ++ [13] systems seem very good candidates for circuits involving evaluations of hash functions like SHA-256.
- STARKs [8] or other hash-based SNARKs like Aurora [7] or Brakedown [25].

4.2 Efficiency and Comparison with State of the Art

Among all those possibilities, maybe the most robust (with less security assumptions) and efficient hash-based instantiation is obtained by using: (i) the Halevi-Micali commitment scheme, (ii) the SPHINCS(+) signature scheme, (iii) Ligerio ++ as the NIZK system.

This is exactly the same instantiation considered in the last paragraphs of (Appendix J of) [19] for their generic construction, which is a bit different to our generic construction: the employed commitment and NIWI proofs are a bit more complicated than ours, but this allows them to prove the Generalized Universally Composable (GUC) security of the resulting blind signature scheme in the non-programmable random oracle model. So in terms of security their scheme is more robust than ours, whose game-based (weaker) security is proved in the (stronger) programmable random oracle model. In terms of efficiency, when using the same basic primitives (commitment scheme, digital signature scheme, interactive ZK system

for C) as [19], our scheme is a bit more efficient than theirs. In particular, and mimicking their asymptotic estimation (end of Appendix J in [19]), the size of our blind signature scheme when instantiated with Halevi-Micali commitment, SPHINCS(+) signature scheme and Ligerio ++ NIZK system would be around 250 KB.

We summarize the state of the art in blind signature schemes with post-quantum security in Table 1. Therein, GB stands for game-based security, GUC stands for Generalized Univesal Composability security, (N)P-ROM stands for (not) Programmable random oracle model, AGAM stands for Algebraic Group Action Model, CRHF stands for collision-resistant hash functions.

Scheme(s)	Size of signatures	PQ Framework	Security Model
[1,12]	22-46 KB	Lattices	P-ROM + GB
[14,4]	4-41 KB	Multivariate Quadratic	P-ROM + GB
[35]	4-5 KB	Coding	P-ROM + AGAM + GB
[27]	4-6 KB	Isogenies	P-ROM + AGAM + GB
[19]	~250 KB	CRHF	NP-ROM + UC
Ours	~250 KB	CRHF	P-ROM + GB

Table 1: State of the art in (signature size of) post-quantum blind signatures.

4.3 Proof of Concept: Implementation of an (Inefficient) Instantiation

We wanted to implement some instantiation to have a proof of concept of our results. We did so by instantiating our generic construction with $\text{Com}(m; r) = H_e(m \parallel r)$, using basic Merkle MSS as the signature scheme S , and using the ZKBoo system [24] (an instantiation of the MPC-in-the-Head paradigm) as the NIZK system Π , because these were the primitives (maybe somehow outdated today) that we knew best. We are currently working on an implementation of the most efficient instantiation of our generic construction, as mentioned in the previous section, using SPHINCS(+) as the signature scheme and (some variant of) Aurora or Ligerio as the NIZK system.

For completeness, we describe in Appendix A the necessary ingredients and the details of the protocols for the resulting instantiation.

Our implementation (written in C) of this scheme can be found [here](#)⁵. This code was built starting from the original implementation of the ZKBoo protocol, from Aarhus University [24], which originally provided a NIZK proof of knowledge of a preimage of a public output of hash function SHA256. The cryptographic functions (SHA256, AES256) are imported from [OpenSSL library](#). The code also uses the [OpenMP](#) API to parallelize proof generation and verification, making the binaries significantly faster.

Parameters. The following parameters were used for the benchmarks presented below (run on Intel 6-Core i5-10400F, 4.30 GHz, 8 GB RAM). The public key pk of the signer S is 32-bytes long as it is an output of SHA256 (the value in the Merkle tree root). The matching secret key sk , which is a PRF key (32 bytes long, we use AES256 as PRF) combined with the Merkle tree index (one integer: 4 bytes). We also chose a 32-byte length for the user’s

⁵ https://github.com/hlouisio/blind_MSS

commitment randomness r . The Merkle tree height was set to $h = 10$, allowing $N = 2^{10} = 1024$ signatures per key pair. Finally, we used $t = 219$ ZKBoo rounds (parallel executions), giving a soundness error of 2^{-128} (considered as the minimal soundness error for a post-quantum construction) and the Fiat-Shamir Transform function H_{FS} is (as announced) SHA256.

Benchmarks. On average, the signer’s binaries (key generation and signing the committed message M) complete in approximately 0.65 seconds per execution. The commitment computed as $\text{SHA256}(\text{SHA256}(m), r)$ by the user U (to be sent to S) involves a random sampling of r and two SHA256 evaluations. Therefore, the execution time mainly depends on the length of m , which determines the number of compression rounds in the SHA256 algorithm. For message lengths between 1 B and 10 MB, it rarely exceeds 0.2 s. The most time-consuming part, obviously, is the computation and verification of the ZKBoo NIZK proof.

Proof/Verification benchmarks. We arbitrarily chose random messages of length 10 kB. Nevertheless, the proof algorithm operates on the public message digest $\text{SHA256}(m)$ (256 bits). For 100 random messages of length 10 kB, the average proof execution time was approximately 7.12 s, producing final signatures (NIZK proofs) σ of about 519 MB. For these message inputs, the verification of the generated proofs took on average 3.25 s.

Operation	Average time (s)	Notes
Commitment computation (user)	< 0.20	For $ m \in [1\text{B}, 10\text{MB}]$
Key generation + Signature Σ (signer)	0.65	
Proof generation (user)	7.12	For 10 kB messages
Proof verification (verifier)	3.25	

Table 2: Average execution times on an Intel 6-Core i5-10400F, 4.30 GHz, 8 GB RAM.

Object	Size
Public key of S (pk)	32 B
Secret key of S (sk)	36 B
MSS signature Σ	16.7 KB
Final signature σ (NIZK proof)	519 MB

Table 3: Sizes of the main cryptographic objects.

5 Conclusions and Future Work

We initiate in this work the research of hash-based blind signature schemes. Due to the lack of algebraic structure in the hash-based setting, it seems the only possibility is to fully

rely on (some adaptation of) Fischlin’s generic construction of blind signatures [23]. Other blind signature schemes with post-quantum security have followed Fischlin’s paradigm in the recent years, based on lattices [1,12], coding [14], quadratic multivariate polynomials [14,15,4] or (similarly and concurrent to us) based on one-way functions [19].

In terms of efficiency, we do not win the intense race that is taking place in the last months: our hash-based blind signature schemes have running times and signature sizes quite worse than those of other schemes (see for instance our Table 1).

However, in terms of (post-quantum) security, our schemes (and those in [19]) rely only on well-studied properties of hash functions, whereas the other schemes must rely (additionally) on the quantum hardness of specific mathematical problems in the settings of lattices, coding theory, multivariate quadratic polynomials or isogenies. Such problems are believed to be quantumly hard today, but maybe this situation can change in the future; in contrast, the post-quantum security offered by hash functions is considered to be more robust.

Another line of future research could be the use of hash-based primitives (commitments, signatures, NIZK proof systems) to instantiate other generic constructions of signatures with additional properties, in particular attribute-based signatures [36] or the (very) related notion of anonymous credentials (as it has been done in [15] for the setting of multivariate polynomials).

Acknowledgments

This work was partially done while Hugo Louiso was visiting Universitat Politècnica de Catalunya (UPC). The authors want to warmly thank Damien Vergnaud for his helpful comments related to this work.

References

1. Agrawal, S., Kirshanova, E., Stehlé, D., Yadav, A.: Practical, round-optimal lattice-based blind signatures. In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*. pp. 39–53. ACM (2022). <https://doi.org/10.1145/3548606.3560650>
2. Ames, S., Hazay, C., Ishai, Y., Venkitasubramaniam, M.: Liger: lightweight sublinear arguments without a trusted setup. *Des. Codes Cryptogr.* **91**(11), 3379–3424 (2023). <https://doi.org/10.1007/S10623-023-01222-8>
3. Bansarkhani, R.E., Misoczki, R.: G-merkle: A hash-based group signature scheme from standard assumptions. In: Lange, T., Steinwandt, R. (eds.) *Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018, Fort Lauderdale, FL, USA, April 9-11, 2018, Proceedings. Lecture Notes in Computer Science*, vol. 10786, pp. 441–463. Springer (2018). https://doi.org/10.1007/978-3-319-79063-3_21, https://doi.org/10.1007/978-3-319-79063-3_21
4. Baum, C., Beckmann, M., Beullens, W., Mukherjee, S., Rechberger, C.: Concretely efficient blind signatures based on vole-in-the-head proofs and the mayo trapdoor. *Cryptology ePrint Archive*, Paper 2026/109 (2026), <https://eprint.iacr.org/2026/109>
5. Baum, C., Beullens, W., Braun, L., de Saint Guilhem, C.D., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: Shorter, tighter, faester: Optimizations and improved (QROM) analysis for vole-in-the-head signatures. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International*

- Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part VI. Lecture Notes in Computer Science, vol. 16005, pp. 124–156. Springer (2025). https://doi.org/10.1007/978-3-032-01887-8_5, https://doi.org/10.1007/978-3-032-01887-8_5
6. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V. Lecture Notes in Computer Science*, vol. 14085, pp. 581–615. Springer (2023). https://doi.org/10.1007/978-3-031-38554-4_19
 7. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent succinct arguments for R1CS. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 11476, pp. 103–128. Springer (2019). https://doi.org/10.1007/978-3-030-17653-2_4
 8. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Zeldovich, N.: Scalable, transparent, and post-quantum secure computational integrity. In: *Advances in Cryptology - CRYPTO 2018, Part II. Lecture Notes in Computer Science*, vol. 10992, pp. 92–120. Springer (2018). https://doi.org/10.1007/978-3-319-96881-0_4
 9. Bernstein, D.J., Hopwood, D., Hülsing, A., Lange, T., Niederhagen, R., Papachristodoulou, L., Schneider, M., Schwabe, P., Wilcox-O’Hearn, Z.: SPHINCS: practical stateless hash-based signatures. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I. Lecture Notes in Computer Science*, vol. 9056, pp. 368–397. Springer (2015). https://doi.org/10.1007/978-3-662-46800-5_15
 10. Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The SPHINCS⁺ signature framework. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*. pp. 2129–2146. ACM (2019). <https://doi.org/10.1145/3319535.3363229>
 11. Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: SPHINCS+ – submission to the 3rd round of the NIST post-quantum project (v3.1). <https://sphincs.org/data/sphincs+-round3-specification.pdf> (2020), accessed: 2025-08-14
 12. Beullens, W., Lyubashevsky, V., Nguyen, N.K., Seiler, G.: Lattice-based blind signatures: Short, efficient, and round-optimal. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*. pp. 16–29. ACM (2023). <https://doi.org/10.1145/3576915.3616613>
 13. Bhadauria, R., Fang, Z., Hazay, C., Venkatasubramanian, M., Xie, T., Zhang, Y.: Liger++: A new optimized sublinear IOP. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) *CCS ’20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*. pp. 2025–2038. ACM (2020). <https://doi.org/10.1145/3372297.3417893>, <https://doi.org/10.1145/3372297.3417893>
 14. Bouillaguet, C., Feneuil, T., Maire, J., Rivain, M., Sauvage, J., Vergnaud, D.: Blinding post-quantum hash-and-sign signatures. *Proceedings of IEEE Symposium on Security and Privacy 2026*, to appear, <https://eprint.iacr.org/2025/895>

15. Bouillaguet, C., Feneuil, T., Maire, J., Rivain, M., Sauvage, J., Vergnaud, D.: Multivariate commitments and signatures with efficient protocols. Cryptology ePrint Archive, Paper 2025/2035 (2025), <https://eprint.iacr.org/2025/2035>
16. Buchmann, J., Dahmen, E., Hülsing, A.: XMSS - A practical forward secure signature scheme based on minimal security assumptions. In: Yang, B. (ed.) Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29 - December 2, 2011. Proceedings. Lecture Notes in Computer Science, vol. 7071, pp. 117–129. Springer (2011). https://doi.org/10.1007/978-3-642-25405-5_8
17. Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Zaverucha, G.: Post-quantum zero-knowledge and signatures from symmetric-key primitives. In: Thuraisingham, B., Evans, D., Malkin, T., Xu, D. (eds.) Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017. pp. 1825–1842. ACM (2017). <https://doi.org/10.1145/3133956.3133997>
18. Chaum, D.: Blind signatures for untraceable payments. In: CRYPTO 1982. LNCS, vol. 196, pp. 199–203. Springer (1983). https://doi.org/10.1007/978-1-4757-0602-4_18
19. Ciampi, M., Monica, P.D., Visconti, I.: Round-optimal guc-secure blind signatures from minimal computational and setup assumptions. Cryptology ePrint Archive, Paper 2026/145 (2026), <https://eprint.iacr.org/2026/145>
20. Coronado, C.: On the security and the efficiency of the Merkle signature scheme. Cryptology ePrint Archive, Paper 2005/192 (2005), <https://eprint.iacr.org/2005/192>
21. Drake, J., Khovratovich, D., Kudinov, M.A., Wagner, B.: Hash-based multi-signatures for post-quantum Ethereum. IACR Commun. Cryptol. **2**(1), 13 (2025). <https://doi.org/10.62056/AEY7QJP10>
22. Feneuil, T., Rivain, M.: Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. J. Cryptol. **38**(3), 28 (2025). <https://doi.org/10.1007/S00145-025-09543-8>
23. Fischlin, M.: Round-optimal composable blind signatures in the common reference string model. In: Dwork, C. (ed.) Advances in Cryptology - CRYPTO 2006, 26th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 2006, Proceedings. Lecture Notes in Computer Science, vol. 4117, pp. 60–77. Springer (2006). https://doi.org/10.1007/11818175_4
24. Giacomelli, I., Madsen, J., Orlandi, C.: Zkboo: Faster zero-knowledge for boolean circuits. In: Holz, T., Savage, S. (eds.) 25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016. pp. 1069–1083. USENIX Association (2016), <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/giacomelli>
25. Golovnev, A., Lee, J., Setty, S.T.V., Thaler, J., Wahby, R.S.: Brakedown: Linear-time and field-agnostic snarks for R1CS. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part II. Lecture Notes in Computer Science, vol. 14082, pp. 193–226. Springer (2023). https://doi.org/10.1007/978-3-031-38545-2_7
26. Halevi, S., Micali, S.: Practical and provably-secure commitment schemes from collision-free hashing. In: Koblitz, N. (ed.) Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Pro-

- ceedings. Lecture Notes in Computer Science, vol. 1109, pp. 201–215. Springer (1996). https://doi.org/10.1007/3-540-68697-5_16, https://doi.org/10.1007/3-540-68697-5_16
27. Hanzlik, L., Lai, Y., Mula, M., Paracucchi, E., Slamanig, D., Tang, G.: Tanuki: New frameworks for (concurrently secure) blind signatures from post-quantum group actions. In: Hanaoka, G., Yang, B. (eds.) *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security*, Melbourne, VIC, Australia, December 8–12, 2025, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 16248, pp. 35–69. Springer (2025). https://doi.org/10.1007/978-981-95-5113-2_2, https://doi.org/10.1007/978-981-95-5113-2_2
 28. Hülsing, A.: W-OTS+ - shorter signatures for hash-based signature schemes. In: Youssef, A.M., Nitaj, A., Hassanien, A.E. (eds.) *Progress in Cryptology - AFRICACRYPT 2013*, 6th International Conference on Cryptology in Africa, Cairo, Egypt, June 22–24, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7918, pp. 173–188. Springer (2013). https://doi.org/10.1007/978-3-642-38553-7_10
 29. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from secure multiparty computation. In: Johnson, D.S., Feige, U. (eds.) *Proceedings of the 39th Annual ACM Symposium on Theory of Computing*, San Diego, California, USA, June 11–13, 2007. pp. 21–30. ACM (2007). <https://doi.org/10.1145/1250790.1250794>
 30. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge proofs from secure multiparty computation. *SIAM Journal on Computing* **39**(3), 1121–1152 (2009). <https://doi.org/10.1137/080725398>
 31. Katz, J., Kolesnikov, V., Wang, X.: Improved non-interactive zero knowledge with applications to post-quantum signatures. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018*, Toronto, ON, Canada, October 15–19, 2018. pp. 525–537. ACM (2018). <https://doi.org/10.1145/3243734.3243805>
 32. Kelsey, J., Lang, N., Lucks, S.: Turning hash-based signatures into distributed signatures and threshold signatures: Delegate your signing capability, and distribute it among trustees. *IACR Commun. Cryptol.* **2**(2), 24 (2025). <https://doi.org/10.62056/A6KSUDY6B>, <https://doi.org/10.62056/a6ksudy6b>
 33. Khaburzaniya, I., Chalkias, K., Lewi, K., Malvai, H.: Aggregating and thresholdizing hash-based signatures using starks. In: Suga, Y., Sakurai, K., Ding, X., Sako, K. (eds.) *ASIA CCS '22: ACM Asia Conference on Computer and Communications Security*, Nagasaki, Japan, 30 May 2022 - 3 June 2022. pp. 393–407. ACM (2022). <https://doi.org/10.1145/3488932.3524128>
 34. Khovratovich, D., Kudinov, M.A., Wagner, B.: At the top of the hypercube - better size-time tradeoffs for hash-based signatures. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part VI. Lecture Notes in Computer Science, vol. 16005, pp. 93–123. Springer (2025). https://doi.org/10.1007/978-3-032-01887-8_4
 35. Lai, Y., Persichetti, E.: LEAF: compact and efficient blind signature from code-based assumptions. *IACR Cryptol. ePrint Arch.* p. 1585 (2025), <https://eprint.iacr.org/2025/1585>
 36. Maji, H.K., Prabhakaran, M., Rosulek, M.: Attribute-based signatures. In: Kiayias, A. (ed.) *Topics in Cryptology - CT-RSA 2011 - The Cryptographers' Track at the RSA Conference 2011*, San Francisco, CA, USA, February 14–18, 2011. Proceedings. Lecture Notes in Computer Science, vol. 6558, pp. 376–392. Springer (2011). https://doi.org/10.1007/978-3-642-19074-2_

37. Merkle, R.C.: A certified digital signature. In: Brassard, G. (ed.) *Advances in Cryptology — CRYPTO' 89 Proceedings*. pp. 218–238. Springer New York, New York, NY (1990)
38. Shor, P.W.: Algorithms for quantum computation: Discrete logarithms and factoring. In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. pp. 124–134. IEEE (1994). <https://doi.org/10.1109/SFCS.1994.365700>
39. Xue, Y., Lu, X., Au, M.H., Zhang, C.: Efficient linkable ring signatures: New framework and post-quantum instantiations. In: García-Alfaro, J., Kozik, R., Choras, M., Katsikas, S.K. (eds.) *Computer Security - ESORICS 2024 - 29th European Symposium on Research in Computer Security*, Bydgoszcz, Poland, September 16-20, 2024, *Proceedings, Part IV*. *Lecture Notes in Computer Science*, vol. 14985, pp. 435–456. Springer (2024). https://doi.org/10.1007/978-3-031-70903-6_22

A Proof of Concept: Implementation of a Hash-Based Blind Signature Scheme

A.1 Ingredients

Hash-based One-Time Signatures. If a signature scheme is secure in a setting where the adversary $\mathcal{F}$ is restricted to make, at most, one signing query before outputting its forgery (m^*, σ^*) , then the signature scheme is said to be *one-time secure*, and one calls it *one-time signature (OTS) scheme*.

One-way functions, in particular a hash function $H : \mathcal{X} \rightarrow \mathcal{Y}$, are enough to design OTS schemes, as shown by [28]. For instance, the following scheme allows to sign messages $m = m_1 m_2 \dots m_k$ with k bits by choosing in Σ .KeyGen k values $x_1, x_2, \dots, x_k \in \mathcal{X}$ independently and at random, that will form the secret key sk . The matching public key pk will contain the k values $y_1, \dots, y_k$, where $y_\ell = H(x_\ell)$ for each $\ell = 1, \dots, k$. In the basic scheme, the signature of a message $m = m_1 m_2 \dots m_k$ is $\sigma = (\sigma_1, \dots, \sigma_k)$ with $\sigma_\ell = x_\ell$ if $m_\ell = 1$ and $\sigma_\ell = y_\ell$ if $m_\ell = 0$. The verification algorithm just checks that $H^{(m_\ell)}(\sigma_\ell) = y_\ell$ for all $\ell = 1, \dots, k$, where $H^{(0)} = \text{Identity}$ and $H^{(1)} = H$.

However, this basic scheme is not secure: an adversary can, for instance, ask for a signature of the message $m = 11 \dots 1$ and use the resulting signature to sign any other message. To prevent this kind of attacks, some *checksum* information must be added to the signature and the keys. It is well known that the optimal size of this checksum information is $\log(k)$, but we will consider here a less efficient (although simpler for our later purposes) solution, with size k for the checksum: the secret key and the public keys will now contain $2k$ elements x_ℓ and $2k$ elements y_ℓ , for $\ell = 1, 2, \dots, 2k$. To sign a k -bit message $m = m_1 m_2 \dots m_k$, the basic scheme is applied to the message $\hat{m} = m \parallel \bar{m} = m_1 m_2 \dots m_k (1 - m_1)(1 - m_2) \dots (1 - m_k)$, resulting in a signature $\sigma = \sigma_1, \dots, \sigma_k, \sigma_{k+1}, \dots, \sigma_{2k}$ of double size.

Merkle Trees: Hash-based Full Signatures. Merkle [37] suggested a way to design a many-times signature scheme from a one-time signature scheme, by using the idea of hash binary trees: if the tree has height h , each of the $N = 2^h$ leaves of the tree, say the leaf ℓ , is related to a pair $(\text{sk}_\ell, \text{pk}_\ell)$ of keys for a one-time signature scheme. Then these N public keys are accumulated with a down-to-up process through the tree, assigning to each node A of the tree the hash of the two values of its two children B_0 (left) and B_1 (right), as $X_A = H(X_{B_0}, X_{B_1})$. The values in the leaves are the public keys pk_ℓ . Finally, the value in

the root of the tree is taken as the public key of the signature scheme. The secret key is the set of N one-time secret keys $\text{sk}_1, \dots, \text{sk}_N$ or (for efficiency) a seed sk that allows the signer to compute such one-time secret keys, for instance through a pseudo-random function: $\text{sk}_\ell = \text{PRF}(\text{sk} \parallel \ell)$.

The resulting signature scheme will allow to securely sign N different messages in a stateful way: the signer must maintain a counter to know how many signatures it has already computed and so which is the following leaf of the tree that must be used.

To sign the ℓ -th message m , using leaf ℓ , the one-time signature scheme is used to compute with sk_ℓ a signature σ_ℓ on message m . If $A_h, A_{h-1}, \dots, A_1, \text{ROOT}$ is the path from leaf ℓ (which is node A_h) to the root of the tree, then the signature must contain the value in node A_h (which is the public key pk_ℓ) and all the values in the sibling nodes to $A_h, A_{h-1}, \dots, A_1$. Therefore, the final signature σ contains the index ℓ , σ_ℓ , pk_ℓ and h hash values.

The verification algorithm first checks that σ_ℓ is a valid signature for m under one-time public key pk_ℓ and then uses the hash values in the signature to recompute (following the unique path defined by index ℓ) the value in the root of the tree and compare it to the public key of the signature scheme.

NIZK from MPC-in-the-Head. Ishai et al. [29,30] gave a nice connection between zero-knowledge proof systems and multi-party computation (MPC), which is known as the multi-party in the head (MPC-in-the-head) paradigm. Recall that a MPC protocol for a function f between n parties, with each party P_i having a secret input x_i , is an interactive protocol in which they exchange messages to finally compute the value $y = f(x_1, \dots, x_n)$. The protocol is secure if nothing else (other than what y leaks) is revealed about the secret inputs x_i .

Now let us suppose that a prover knows a secret witness ω such that $C(\omega) = 1$ for some circuit (or function) C . The MPC-in-the-head paradigm makes such a prover distribute ω in n shares $\omega_1, \dots, \omega_n$ and then run (in its head) a MPC protocol where these n parties jointly compute $f(\omega_1, \dots, \omega_n) = C(\omega)$. The prover computes commitments to the views of all these n parties and sends such commitments to the verifier, in the first interaction. The verifier sends as its challenge a subset A of the n parties. Finally, the prover sends the real views of the parties in A . The verifier accepts if the opened views match the previous commitments, and if these views are consistent with an execution of the MPC protocol that ends with an output $y = 1$.

This is an interactive zero-knowledge proof system for circuit C , that can be turned into a NIZK for C by applying the Fiat-Shamir transformation, since the only message from the verifier is the random selection of the subset A (which can be encoded as a random element in $\{0, 1\}^n$).

A.2 Details of the Scheme

As we need randomness to generate our Merkle tree and to preserve the zero-knowledge property when opening some views, we use AES256 in counter mode (AES-CTR) as our pseudorandom function (PRF). SHA256 was chosen as the hash functions H , H_c , H_{FS} , as it is commonly used in most hash-based signature schemes (WOTS, SPHINCS+, XMSS, etc.).

U sends commitment of m to S : the commitment to a message m is computed as $M = \text{Com}(m; r) = H_c(m \parallel r)$, for some (quantum) secure hash function $H_c : \{0, 1\}^* \rightarrow \{0, 1\}^k$, taking as r a random string of 32 bytes (256 bits).

S signs M with MSS and sends Σ to U : S will sign this commitment M with a stateful Merkle signature scheme MSS [20], instantiated by having as OTS a binary (and not-optimized, in terms of checksum) version of WOTS [28], that has already been sketched in Section A.1.

We will denote with $H : \{0, 1\}^* \rightarrow \{0, 1\}^k$ the hash function used everywhere in this MSS signature scheme. k is in particular the size of the public-key pk . The height of the tree is h , and so the scheme can be used to sign $N = 2^h$ different messages, in a stateful way.

The (dynamic) secret key of the signer consists of a random seed sk_{seed} (of size k), for a pseudo-random function PRF with output space $\{0, 1\}^k$, and the index idx that recalls the number of signatures that have been produced, initially $\text{idx} = 0$. The public key pk of S is computed as follows:

- For each leaf $\ell = 0, 1, \dots, N - 1$ in the tree:
 1. For $i = 0, 1, \dots, 2k - 1$, compute the values $\text{sk}_{\ell,i} = \text{PRF}(\text{sk}_{\text{seed}}, \ell, i)$ and $\text{pk}_{\ell,i} = H(\text{sk}_{\ell,i})$, and then define $\text{sk}_\ell = (\text{sk}_{\ell,0}, \text{sk}_{\ell,1}, \dots, \text{sk}_{\ell,2k-1})$ and $\text{pk}_\ell = (\text{pk}_{\ell,0}, \text{pk}_{\ell,1}, \dots, \text{pk}_{\ell,2k-1})$ as the OTS secret and public keys of the leaf ℓ . Each $\text{sk}_{\ell,i}$ and $\text{pk}_{\ell,i}$ is k bits long.
 2. The value of the leaf ℓ node is defined as $\text{leaf_node}_\ell = H(\text{pk}_\ell)$.
- Compute the values in the remaining nodes of the tree (not leaves, so they are parents with two children each) as $\text{upper_node} = H(\text{left_child} \parallel \text{right_child})$.
- The value root_node computed in the root of the tree is defined to be the public key pk of S .

When S receives the k bits $M = H_c(m \parallel r)$ from U , let us assume that $\text{idx} = \ell$ and so S will use the leaf ℓ to sign the doubled message $\hat{M} = (M \parallel \overline{M}) = \hat{m}_0 \hat{m}_1 \dots \hat{m}_{2k-1} \in \{0, 1\}^{2k}$, obtaining a signature $\hat{\sigma} = \hat{\sigma}_0 \hat{\sigma}_1 \dots \hat{\sigma}_{2k-1}$, where

$$\sigma_i = \begin{cases} \text{sk}_{\ell,i} & \text{if } \hat{m}_i = 0 \\ \text{pk}_{\ell,i} = H(\text{sk}_{\ell,i}) & \text{if } \hat{m}_i = 1 \end{cases}$$

Other than $\hat{\sigma}$, the final signature Σ must contain the description of the path from the used leaf ℓ to the root of the tree, and the values in the siblings of the nodes in the path. Specifically, if $A_h, A_{h-1}, \dots, A_1, A_0 = \text{root}$ is the path from the leaf ℓ (which is node A_h) to the root of the tree, being A_j a child of A_{j-1} for $j \in \{h, h-1, \dots, 1\}$, we define the string of bits $\text{path} = (i_h, i_{h-1}, \dots, i_1) \in \{0, 1\}^h$ as: $i_j = 1$ if A_j is the right sibling, $i_j = 0$ if A_j is the left sibling. For each node A_j in the path, $j \in \{h, h-1, \dots, 1\}$ we define as B_j the value stored in its sibling node.

The final signature that S sends to U is $\Sigma = (\hat{\sigma}, \text{path}, B_h, B_{h-1}, \dots, B_1)$.

U blinds Σ to produce the final σ : now U needs to build a NIZK proof of knowledge π of (r, Σ) such that:

$$\text{MSS.Vfy}(\text{pk}, \Sigma, (H_c(m \parallel r) \parallel \overline{H_c(m \parallel r)})) = 1$$

We will use ZKBoo [24], a particular instantiation of the MPC-in-the-head paradigm, to build such a NIZK proof. Let us describe the verification circuit C that is built in zero-knowledge via the MPC-in-the-head paradigm.

It takes as *public* inputs the message m to be signed and the public key pk of signer S , for the employed MSS signature scheme. As *witness* (secret) inputs, circuit C takes the commitment randomness r and the MSS signature $\Sigma = (\hat{\sigma}, \text{path}, B_h, B_{h-1}, \dots, B_1)$.

The circuit does the following operations.

1. Recompute the commitment and its doubled version: $\hat{M} = H_c(m \parallel r) \parallel \overline{H_c(m \parallel r)} = \hat{m}_0 \hat{m}_1 \dots \hat{m}_{2k-1} \in \{0, 1\}^{2k}$.
2. Recompute the WOTS public key of the corresponding leaf: for each $i \in \{0, 1, \dots, 2k-1\}$, the signature component $\hat{\sigma}_i$ is either $\text{sk}_{\ell,i}$ (when $\hat{m}_i = 0$) or its hash $\text{pk}_{\ell,i} = H(\text{sk}_{\ell,i})$ (when $\hat{m}_i = 1$). The idea is that the circuit C will recompute all the public key $\text{pk}_\ell = (\text{pk}_{\ell,0}, \dots, \text{pk}_{\ell,2k-1})$ from the signature $\hat{\sigma} = \hat{\sigma}_0 \dots \hat{\sigma}_{2k-1}$, as follows:

$$\text{pk}_{\ell,i} = (\hat{m}_i \wedge \hat{\sigma}_i) \oplus (\overline{\hat{m}_i} \wedge H(\hat{\sigma}_i))$$

where $\wedge$ between a bit b and a string of bits s denotes the result of doing an AND between b and each bit of s , and $\overline{b} = 1 - b$ for any bit b .

3. Recompute the values in the nodes of the path, from the leaf to the root: the value in the leaf ℓ corresponding to that signature is computed from the WOTS public key $\text{pk}_\ell = (\text{pk}_{\ell,0}, \dots, \text{pk}_{\ell,2k-1})$ as $A_h = \text{leaf_node} = H(\text{pk}_\ell)$. The following node in the path, with value A_{h-1} , is the parent of siblings A_h and B_h (the value of B_h is contained in Σ). The information of which is the left child and which is the right child is in the bit i_h of string $\text{path} = (i_h, i_{h-1}, \dots, i_1) \in \{0, 1\}^h$: in general, if $i_j = 0$, then A_j is the left child and B_j is the right child of A_{j-1} . Therefore, the circuit C can iteratively compute these values as

$$A_{j-1} = H((\overline{i_j} \wedge A_j) \oplus (i_j \wedge B_j), (i_j \wedge A_j) \oplus (\overline{i_j} \wedge B_j))$$

where again $b \wedge s$ is an AND between a bit and all the bits of the string s .

After h iterations the circuit obtains the value $A_0 = \text{root_node}$ in the root of the tree, and finally checks if this value is equal to the public input pk (the public key of the signer S).

The circuit C outputs 1 if and only if $A_0 = \text{pk}$.

ZKBoo, MPC-in-the-Head. In ZKBoo Σ -protocol [24], the MPC-in-the-head paradigm is implemented with $n = 3$ parties. This means that every secret input bit b of the circuit C is distributed in three random shares b_1, b_2, b_3 such that $b = b_1 \oplus b_2 \oplus b_3$, and later every arithmetic gate of the circuit is evaluated in a MPC way, to produce shares of the output from shares of the inputs.

When the execution of the whole circuit is finished (in the head of the prover), each party $P_i \in \{P_1, P_2, P_3\}$ has its corresponding view view_i of the execution of the protocol. A special part of this view is a seed k_i that party P_i uses to produce all the local randomness needed by the MPC evaluation of C .

The first message of the Σ -protocol, from the prover to the verifier, contains commitments $\hat{C}_i = \text{Com}(k_i \parallel \text{view}_i ; r_i)$ to the views of all three parties, $i = 1, 2, 3$. The second message of the Σ -protocol, from the verifier to the prover, is a random challenge $e \in \{1, 2, 3\}$. In the third message, in the original description of ZKBoo [24], the prover reveals r_i, k_i, view_i for two parties, $i = e$ and $i = e + 1 \pmod 3$.

The final verification consists in checking that the two commitments $\hat{C}_i$ are correct, for $i = e$ and $i = e + 1 \pmod 3$, and that these two views are mutually consistent as part of an execution of the MPC protocol that evaluates circuit C with an output of 1.

Fiat-Shamir Transform for NIZK Proof. Following the standard Fiat-Shamir paradigm and the concrete instantiation of ZKBoo [24], we make this protocol non-interactive in the random-oracle model.

Let t be the number of repetitions of the basic ZKBoo protocol that we use to obtain the desired soundness error, and let

$$H_{\text{FS}} : \{0, 1\}^* \rightarrow \{1, 2, 3\}^t$$

be a hash function modeled as a random oracle (instantiated in practice with SHA256, as in [24]): we interpret the hash output as a bitstring, scan it in consecutive pairs of bits, map 00, 01, 10 respectively to challenges 1, 2, 3 and discard pair 11 via rejection sampling; if one hash call does not yield t valid challenges, we re-apply SHA256 to the previous input concatenated with a counter and continue this extraction process until t challenges are obtained. For a fixed instance (m, pk) , the prover first executes the *Commit* step of all t parallel copies of the Σ -protocol and obtains the first message

$$a = (a^{(1)}, \dots, a^{(t)}),$$

where each $a^{(j)}$ contains the commitments to the three views $(k_i^{(j)}, \text{view}_i^{(j)})$, $i \in \{1, 2, 3\}$, of the j -th MPC-in-the-head execution.

Instead of receiving t independent random challenges from the verifier, the prover computes a vector of challenges

$$e = (e_1, \dots, e_t) \leftarrow H_{\text{FS}}(H(m), \text{pk}, a) \in \{1, 2, 3\}^t$$

and, for each repetition $j \in \{1, \dots, t\}$, opens the two views indexed by e_j and $e_j + 1 \bmod 3$, exactly as in the interactive protocol. The resulting non-interactive proof is the transcript

$$\pi = (a, z),$$

where z collects all the opened seeds and commitments openings for the t repetitions.

Given (m, pk) and a purported proof $\pi = (a, z)$, the verifier deterministically recomputes $e = H_{\text{FS}}(H(m), \text{pk}, a)$ and then runs the original ZKBoo verification algorithm on all t repetitions, using these challenges. By the analysis of ZKBoo, and since H_{FS} is modeled as a random oracle, the resulting protocol is a NIZK argument for (r, Σ) with soundness error $(2/3)^t$ and the same zero-knowledge guarantees as the underlying Σ -protocol.

ZKBoo with Reduced Proof Size. Following the optimization O6 of ZKB++ [17], we slightly modify the verification phase of the above Σ -protocol in order to reduce the size of the opened transcripts.

In the original ZKBoo protocol, when the verifier opens the two parties P_e and P_{e+1} , it receives for both of them the seed k_i and the full view view_i , and then explicitly checks gate-by-gate consistency of these two views (in particular, at the gates where P_e and P_{e+1} have to communicate).

We instead proceed as follows. For the challenged pair (P_e, P_{e+1}) the prover reveals:

- the commitment randomness r_e, r_{e+1} ,
- the seeds k_e, k_{e+1} ,
- the full view view_{e+1} ,
- and, from view_e , only the share of the private input (r, Σ) .

Given this information, the public description of the circuit C and its public inputs, the verifier can locally recompute all the remaining values of view_e , and then simply check that the commitments to (k_e, view_e) and $(k_{e+1}, \text{view}_{e+1})$ open correctly. By the binding property of the commitment scheme, these commitments can only verify if view_e is consistent with a correct MPC execution of C , which preserves the soundness guarantees of ZKBoo while reducing the proof size.